Laboratorio
Gestión Completa CRUD con Roles (Ionic, Angular Standalone, Supabase)

Módulo Profesional: Desarrollo de Interfaces (Código 0488)
Ciclo Formativo: Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM2)
Alineación Curricular Comunidad de Madrid:

- **RA 1:** Confecciona interfaces gráficas de usuario utilizando herramientas visuales y librerías de componentes.
  - *CE 1.c:* Se han establecido los mecanismos de enlace entre los datos del modelo y la vista (Data Binding bidireccional reactivo).
- **RA 3:** Crea componentes visuales personalizados evaluando y gestionando eventos.
  - *CE 3.d:* Se han gestionado eventos de autenticación y modificado la interfaz en función de los roles de usuario.
- **RA 4:** Diseña interfaces usables y accesibles.
  - *CE 4.a:* Se han aplicado principios de diseño responsivo (Uso de ion-list y ion-grid).

1. Base Teórica Técnica

El desarrollo de interfaces modernas exige la implementación de **Control de Acceso Basado en Roles (RBAC)** tanto en el cliente (UI) como en el servidor (Backend). En arquitecturas *Serverless* con **Supabase**, la seguridad no recae en la aplicación Angular, sino en las **Políticas de Seguridad a Nivel de Fila (RLS)** de PostgreSQL.
La interfaz desarrollada con Ionic y Angular Standalone tiene un doble propósito:

1. **Formularios Reactivos (ReactiveFormsModule):** Proveen una validación síncrona en cliente, previniendo peticiones malformadas y mejorando la UX.
2. **Renderizado Condicional (@if):** Adapta la interfaz (ocultando botones de inserción, actualización y borrado) evaluando en tiempo real el perfil del usuario autenticado.

2. Implementación Paso a Paso

Paso 2.1: Configuración del Backend (SQL en Supabase)

Ejecuta el siguiente script en el **SQL Editor** de tu proyecto Supabase para crear las tablas, el control de roles y aplicar el RLS estricto.

```
SQL
-- 1. Crear enumeración de roles
CREATE TYPE public.user_role AS ENUM ('admin', 'user');

-- 2. Crear tabla de perfiles (vinculada a autenticación)
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  role public.user_role DEFAULT 'user' NOT NULL
);

-- 3. Crear tabla de clientes
CREATE TABLE public.clients (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  nombre TEXT NOT NULL,
  empresa TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. Habilitar RLS
ALTER TABLE public.clients ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- 5. Políticas para Perfiles (Lectura propia)
CREATE POLICY "Leer perfil propio" ON public.profiles FOR SELECT USING (auth.uid() =
id);

-- 6. Políticas para Clientes (RBAC)
-- Lectura: Todos los usuarios autenticados
CREATE POLICY "Lectura para autenticados" ON public.clients
FOR SELECT TO authenticated USING (true);

-- Escritura (Insert, Update, Delete): SOLO Administradores
CREATE POLICY "Escritura solo para admins" ON public.clients
FOR ALL TO authenticated
USING (EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin'))
WITH CHECK (EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role =
'admin'));
```

(Nota: Deberás crear al menos un usuario manualmente en Supabase Auth e insertarle el rol 'admin' en la tabla profiles para probar).

Paso 2.2: Creación del Proyecto Ionic en VS Code

Abre la terminal integrada en Visual Studio Code y ejecuta los comandos para un proyecto Standalone moderno (Angular 17+ compatible con arquitecturas futuras):

```
Bash
# Instalar CLI de Ionic (si no lo tienes)
npm install -g @ionic/cli

# Crear proyecto Standalone
ionic start dam-crm-app blank --type=angular --standalone

# Acceder al directorio
cd dam-crm-app

# Instalar SDK de Supabase
npm install @supabase/supabase-js
```

Añade tus credenciales en src/environments/environment.ts:

```
TypeScript
export const environment = {
  production: false,
  supabaseUrl: 'https://TU-PROYECTO.supabase.co',
  supabaseKey: 'TU-ANON-KEY'
};
```

Paso 2.3: Interfaz del Modelo (src/app/models/client.model.ts)

```
TypeScript
export interface Client {
  id?: string;
  nombre: string;
  empresa: string;
  email: string;
  created_at?: string;
}
```

Paso 2.4: Servicio de Supabase (src/app/services/supabase.service.ts)
Generamos un servicio único para gestionar Auth y la tabla Clients.

Bash
```
ionic g service services/supabase
```

TypeScript
```
import { Injectable, signal } from '@angular/core';
import { createClient, SupabaseClient, User } from '@supabase/supabase-js';
import { environment } from '../../environments/environment';
import { Client } from '../models/client.model';

@Injectable({
  providedIn: 'root'
})
export class SupabaseService {
  private supabase: SupabaseClient;

  // Señales reactivas para el estado global
  public currentUser = signal<User | null>(null);
  public userRole = signal<'admin' | 'user' | null>(null);

  constructor() {
    this.supabase = createClient(environment.supabaseUrl, environment.supabaseKey);
    this.initSession();
  }

  // --- AUTENTICACIÓN Y ROLES ---
  private async initSession() {
    const { data: { session } } = await this.supabase.auth.getSession();
    if (session?.user) {
      this.currentUser.set(session.user);
      await this.loadRole(session.user.id);
    }
  }

  async login(email: string, password: string) {
    const { data, error } = await this.supabase.auth.signInWithPassword({ email, password });
    if (error) throw error;
    if (data.user) {
      this.currentUser.set(data.user);
      await this.loadRole(data.user.id);
    }
```

```
  }

  private async loadRole(userId: string) {
    const { data, error } = await this.supabase
      .from('profiles')
      .select('role')
      .eq('id', userId)
      .single();
    if (!error && data) {
      this.userRole.set(data.role);
    }
  }

  // --- CRUD CLIENTES ---
  async getClients(): Promise<Client[]> {
    const { data, error } = await this.supabase.from('clients').select('*').order('created_at', {
ascending: false });
    if (error) throw error;
    return data || [];
  }

  async createClient(client: Client): Promise<void> {
    const { error } = await this.supabase.from('clients').insert([{
      nombre: client.nombre,
      empresa: client.empresa,
      email: client.email
    }]);
    if (error) throw error;
  }

  async updateClient(id: string, client: Partial<Client>): Promise<void> {
    const { error } = await this.supabase.from('clients').update(client).eq('id', id);
    if (error) throw error;
  }

  async deleteClient(id: string): Promise<void> {
    const { error } = await this.supabase.from('clients').delete().eq('id', id);
    if (error) throw error;
  }
}
```

Paso 2.5: Lógica de la Página (Controlador) (src/app/home/home.page.ts)
Utilizamos FormBuilder para el formulario reactivo y gestionamos la lógica de CUD utilizando IonItemSliding (deslizamiento nativo en móviles).

```
TypeScript
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { SupabaseService } from '../services/supabase.service';
import { Client } from '../models/client.model';
import {
  IonHeader, IonToolbar, IonTitle, IonContent, IonList, IonItem, IonLabel,
  IonItemSliding, IonItemOptions, IonItemOption, IonIcon, IonButton,
  IonInput, IonCard, IonCardContent, IonCardHeader, IonCardTitle, IonToast
} from '@ionic/angular/standalone';
import { addIcons } from 'ionicons';
import { createOutline, trashOutline } from 'ionicons/icons';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  standalone: true,
  imports: [
    CommonModule, ReactiveFormsModule, IonHeader, IonToolbar, IonTitle, IonContent,
    IonList, IonItem, IonLabel, IonItemSliding, IonItemOptions, IonItemOption,
    IonIcon, IonButton, IonInput, IonCard, IonCardContent, IonCardHeader, IonCardTitle,
IonToast
  ],
})
export class HomePage implements OnInit {
  private supabaseService = inject(SupabaseService);
  private fb = inject(FormBuilder);

  clients: Client[] = [];
  clientForm: FormGroup;
  editingClientId: string | null = null;
  toastMessage = '';
  isToastOpen = false;

  // Señales expuestas a la vista
  isAdmin = this.supabaseService.userRole;
```

```
constructor() {
  addIcons({ createOutline, trashOutline });
  this.clientForm = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3)]],
    empresa: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]]
  });
}

async ngOnInit() {
  // IMPORTANTE: En un entorno real, hacer login antes de cargar
  // Para simplificar esta demo, asumimos que el servicio ya está logueado
  // await this.supabaseService.login('admin@test.com', '123456');
  await this.loadClients();
}

async loadClients() {
  try {
    this.clients = await this.supabaseService.getClients();
  } catch (e: any) {
    this.showToast('Error cargando clientes: ' + e.message);
  }
}

async saveClient() {
  if (this.clientForm.invalid) return;

  try {
    const formValue = this.clientForm.value;
    if (this.editingClientId) {
      await this.supabaseService.updateClient(this.editingClientId, formValue);
      this.showToast('Cliente actualizado');
    } else {
      await this.supabaseService.createClient(formValue);
      this.showToast('Cliente insertado');
    }
    this.resetForm();
    await this.loadClients();
  } catch (e: any) {
    this.showToast('Error de guardado (¿Eres admin?): ' + e.message);
  }
}
```

```
  editClient(client: Client) {
    this.editingClientId = client.id!;
    this.clientForm.patchValue({
      nombre: client.nombre,
      empresa: client.empresa,
      email: client.email
    });
  }

  async deleteClient(id: string) {
    try {
      await this.supabaseService.deleteClient(id);
      this.showToast('Cliente eliminado');
      await this.loadClients();
    } catch (e: any) {
      this.showToast('Error eliminando: ' + e.message);
    }
  }

  resetForm() {
    this.clientForm.reset();
    this.editingClientId = null;
  }

  setToastOpen(isOpen: boolean) {
    this.isToastOpen = isOpen;
  }

  showToast(msg: string) {
    this.toastMessage = msg;
    this.isToastOpen = true;
  }
}
```

Paso 2.6: La Vista (HTML) (src/app/home/home.page.html)
Uso del Control Flow moderno de Angular (@if) y componentes Ionic para una UI limpia.

HTML

```
<ion-header>
  <ion-toolbar color="primary">
    <ion-title>DAM CRM - Gestión de Clientes</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">

  <!-- FORMULARIO REACTIVO (Solo visible para admins) -->
  @if (isAdmin() === 'admin') {
    <ion-card>
      <ion-card-header>
        <ion-card-title>{{ editingClientId ? 'Editar Cliente' : 'Nuevo Cliente' }}</ion-card-title>
      </ion-card-header>
      <ion-card-content>
        <form [formGroup]="clientForm" (ngSubmit)="saveClient()">
          <ion-item>
            <ion-label position="stacked">Nombre</ion-label>
            <ion-input type="text" formControlName="nombre"></ion-input>
          </ion-item>
          <ion-item>
            <ion-label position="stacked">Empresa</ion-label>
            <ion-input type="text" formControlName="empresa"></ion-input>
          </ion-item>
          <ion-item>
            <ion-label position="stacked">Email</ion-label>
            <ion-input type="email" formControlName="email"></ion-input>
          </ion-item>

          <div class="ion-margin-top">
            <ion-button type="submit" expand="block" [disabled]="clientForm.invalid">
              {{ editingClientId ? 'Actualizar' : 'Guardar' }}
            </ion-button>
            @if (editingClientId) {
              <ion-button type="button" fill="clear" expand="block" color="medium"
(click)="resetForm()">
                Cancelar
              </ion-button>
            }
```

```
        </div>
      </form>
    </ion-card-content>
  </ion-card>
}

<!-- TABLA / LISTA DE CLIENTES -->
<h3>Listado de Clientes</h3>
<ion-list>
  @for (client of clients; track client.id) {
    <ion-item-sliding>
      <ion-item>
        <ion-label>
          <h2>{{ client.nombre }}</h2>
          <p>{{ client.empresa }} | {{ client.email }}</p>
        </ion-label>
      </ion-item>

      <!-- Opciones al deslizar (Solo Administradores) -->
      @if (isAdmin() === 'admin') {
        <ion-item-options side="end">
          <ion-item-option color="primary" (click)="editClient(client)">
            <ion-icon slot="icon-only" name="create-outline"></ion-icon>
          </ion-item-option>
          <ion-item-option color="danger" (click)="deleteClient(client.id!)">
            <ion-icon slot="icon-only" name="trash-outline"></ion-icon>
          </ion-item-option>
        </ion-item-options>
      }
    </ion-item-sliding>
  } @empty {
    <ion-item>
      <ion-label color="medium">No hay clientes registrados.</ion-label>
    </ion-item>
  }
</ion-list>

<ion-toast
  [isOpen]="isToastOpen"
  [message]="toastMessage"
  duration="2000"
  (didDismiss)="setToastOpen(false)">
</ion-toast>
```

</ion-content>